

將 BigQuery 與 Node.js 搭配使用

來源：<https://codelabs.developers.google.com/codelabs/cloud-bigquery-nodejs?hl=zh-tw>

步驟數：11

在本程式碼研究室中，您將瞭解如何搭配 Node.js 用戶端程式庫使用 BigQuery

目錄

1. 總覽
2. 設定和需求
3. 啟用 BigQuery API
4. 驗證 API 要求
5. 設定存取控管機制
6. 安裝 Node.js 專用的 BigQuery 用戶端程式庫
7. 查詢莎士比亞的作品
8. 查詢 GitHub 資料集
9. 快取和統計資料
10. 將資料載入 BigQuery
11. 恭喜！

步驟 1 · 約 2 分鐘

1. 總覽

[BigQuery](#) 是 Google 推出的 PB 級全代管低成本數據分析資料倉儲。由於 BigQuery 沒有可管理的基礎架構，更不需要資料庫管理員，因此免除了營運方面的工作，讓您可專心分析資料以找出有意義的結果、使用熟悉的 SQL，以及利用我們的即付即用模型。

在本程式碼研究室中，您將使用 [Google Cloud BigQuery 用戶端程式庫](#)，透過 Node.js 查詢 [BigQuery 公開資料集](#)。

課程內容

- 如何使用 Cloud Shell
- 如何啟用 BigQuery API
- 如何驗證 API 要求
- 如何安裝 Node.js 專用的 BigQuery 用戶端程式庫
- 如何查詢莎士比亞的作品
- 如何查詢 GitHub 資料集
- 如何調整快取和顯示統計資料

軟硬體需求

- Google Cloud Platform 專案
- [Chrome](#) 或 [Firefox](#) 瀏覽器
- 熟悉 Node.js 的使用方式

問卷調查

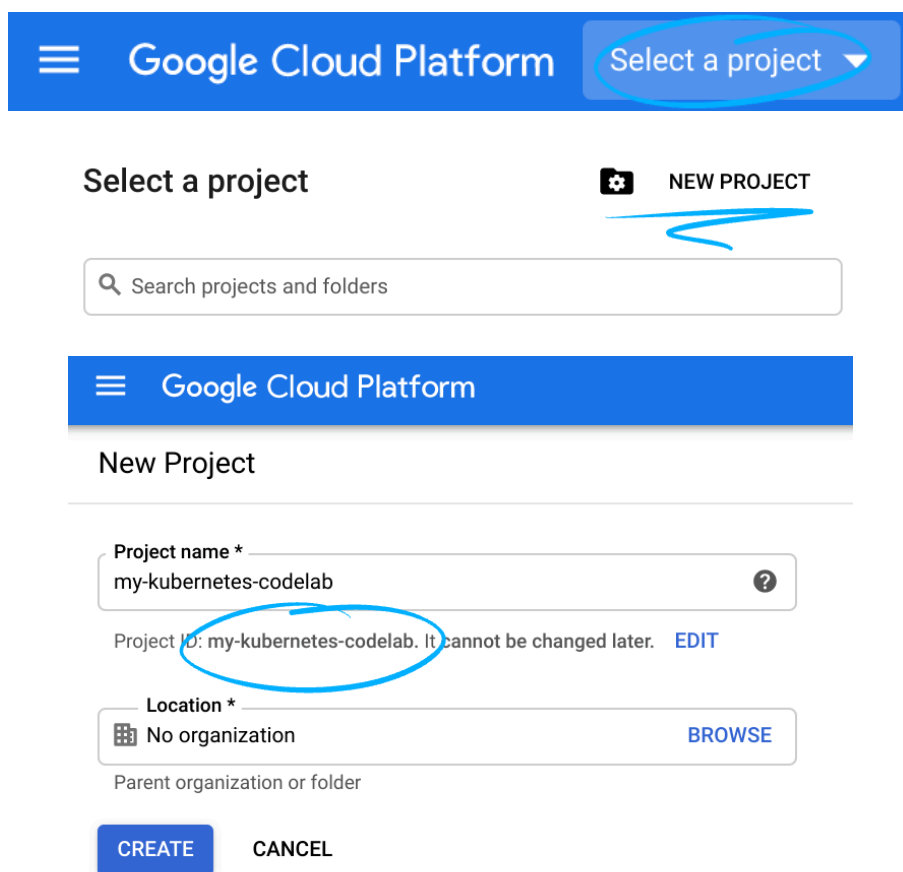
步驟 2 · 約 5 分鐘

2. 設定和需求

自修實驗室環境設定

1. 登入 [Cloud 控制台](#)，建立新專案或重複使用現有專案。(如果沒有 Gmail 或 G Suite 帳戶，請先[建立帳戶](#))。

注意：記住以下網址，即可輕鬆存取 Cloud 控制台：console.cloud.google.com。



The screenshot shows the Google Cloud Platform interface for creating a new project. The top bar is blue with the Google Cloud Platform logo and a 'Select a project' dropdown. Below this, the 'Select a project' dialog is open. It has a search bar for projects and folders. The 'Project name' field is filled with 'my-kubernetes-codelab' and has a question mark icon. The 'Project ID' field shows 'my-kubernetes-codelab' and a note that it cannot be changed later. The 'Location' is set to 'No organization'. The 'CREATE' button is highlighted.

請記住專案 ID，這是所有 Google Cloud 專案中不重複的名稱 (上述名稱已遭占用，因此不適用於您，抱歉！)。本程式碼研究室稍後會將其稱為 `PROJECT_ID`。

注意：如果您使用 Gmail 帳戶，可以將預設位置設為「沒有機構」。如果您使用 G Suite 帳戶，請選擇適合貴機構的位置。

2. 接著，您必須在 Cloud 控制台中[啟用帳單](#)，才能使用 Google Cloud 資源。

完成本程式碼研究室的費用應該不高，甚至完全免費。請務必按照「清除」部分的指示操作，瞭解如何停用資源，避免在本教學課程結束後繼續產生帳單費用。Google Cloud 新使用者可參加[價值\\$300 美元的免費試用計畫](#)。

啟動 Cloud Shell

雖然可以從筆電遠端操作 Cloud SDK 指令列工具，但在本程式碼研究室中，您將使用 [Google Cloud Shell](#)，這是雲端執行的指令列環境。

啟用 Cloud Shell

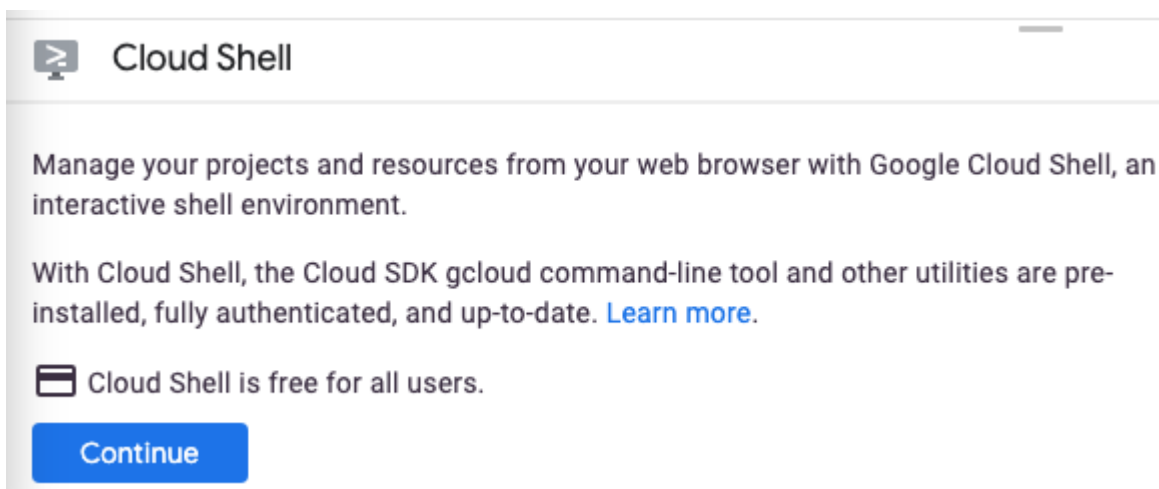
1. 在 Cloud 控制台，點選「啟用 Cloud Shell」圖示



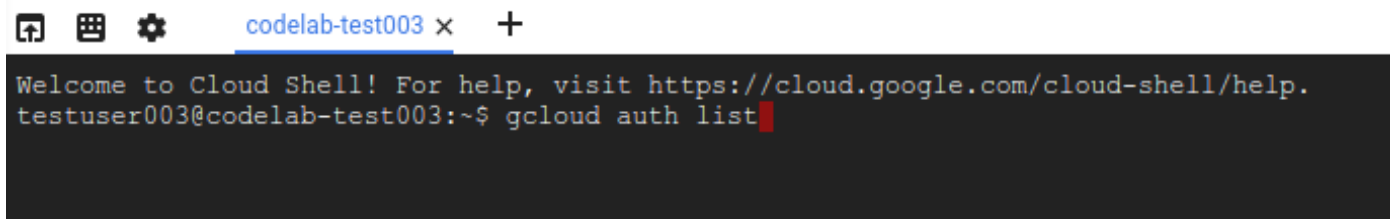
。



如果您是首次啟動 Cloud Shell，系統會顯示中繼畫面 (位於摺疊式選單下方)，說明這個指令列環境。點選「繼續」後，這則訊息日後就不會再出現。以下是這個初次畫面的樣子：



佈建並連至 Cloud Shell 預計只需要幾分鐘。



這部虛擬機器搭載您需要的所有開發工具，並提供永久的 5GB 主目錄，而且可在 Google Cloud 運作，大幅提升網路效能並強化驗證功能。本程式碼研究室幾乎所有工作都可在瀏覽器或 Chromebook 上完成。

連線至 Cloud Shell 後，您應會發現自己通過驗證，且專案已設為您的專案 ID。

2. 在 Cloud Shell 中執行下列指令，確認您已通過驗證：

```
gcloud auth list
```

指令輸出

Credentialed Accounts

ACTIVE ACCOUNT

* <my_account>@<my_domain.com>

To set the active account, run:

```
$ gcloud config set account `ACCOUNT`
```

注意： `gcloud` 指令列工具是 Google Cloud 中功能強大的整合式指令列工具，並已預先安裝於 Cloud Shell。你會發現它支援 Tab 鍵自動完成功能。詳情請參閱 [gcloud 指令列工具總覽](#)。

```
gcloud config list project
```

指令輸出

```
[core]
project = <PROJECT_ID>
```

如未設定，請輸入下列指令手動設定專案：

```
gcloud config set project <PROJECT_ID>
```

指令輸出

```
Updated property [core/project].
```

步驟 3 · 約 2 分鐘

3. 啟用 BigQuery API

所有 Google Cloud 專案預設都會啟用 BigQuery API。您可以在 Cloud Shell 中執行下列指令，確認是否為 true：

```
gcloud services list
```

您應該會看到 BigQuery 列於其中：

NAME	TITLE
bigquery-json.googleapis.com	BigQuery API
...	

如果 BigQuery API 未啟用，您可以在 Cloud Shell 中使用下列指令啟用：

```
gcloud services enable bigquery-json.googleapis.com
```

注意：如果這項指令發生錯誤，請確認目前的專案 ID 與程式碼研究室專案 ID 相符。

使用下列指令找出 Cloud Shell 目前使用的專案 ID：

```
gcloud info | grep "project"
```

如果專案 ID 不正確，請使用下列指令來使用正確的專案 ID：

```
gcloud config set project <PROJECT_ID>
```

將 `<PROJECT_ID>` 替換為正確的專案 ID。

4. 驗證 API 要求

如要向 BigQuery API 提出要求，必須使用「服務帳戶」。「服務帳戶」屬於您的專案，Google BigQuery Node.js 用戶端程式庫會使用此帳戶提出 BigQuery API 要求。服務帳戶與其他使用者帳戶一樣，都是以電子郵件地址表示。在本節中，您將使用 *Cloud SDK* 建立服務帳戶，然後建立以服務帳戶身分進行驗證所需的憑證。

請先使用 `PROJECT_ID` 設定環境變數，本程式碼研究室全程都會使用這個環境變數：

```
export GOOGLE_CLOUD_PROJECT=$(gcloud config get-value core/project)
```

接著，請使用下列方式建立新的服務帳戶，用以存取 BigQuery API：

```
gcloud iam service-accounts create my-bigquery-sa --display-name "my bigquery codelab service account"
```

接著，建立 Node.js 程式碼使用的憑證，用以登入新的服務帳戶。輸入下列指令，建立憑證並儲存為 JSON 檔案「~/key.json」：

```
gcloud iam service-accounts keys create ~/key.json --iam-account my-bigquery-sa@${GOOGLE_CLOUD_PROJECT}.iam.gserviceaccount.com
```

最後，設定 **`GOOGLE_APPLICATION_CREDENTIALS`** 環境變數，BigQuery API C# 程式庫 (下一個步驟會介紹) 會使用這個變數尋找憑證。環境變數應設為您所建立 JSON 憑證檔案的完整路徑。使用下列指令設定環境變數：

```
export GOOGLE_APPLICATION_CREDENTIALS="/home/${USER}/key.json"
```

您可以進一步瞭解如何[驗證 BigQuery API](#)。

5. 設定存取控管機制

BigQuery 使用 Identity and Access Management (IAM) 管理資源存取權。BigQuery 有許多預先定義的角色 (使用者、資料擁有者、資料檢視者等)，您可以指派給上一個步驟中建立的服務帳戶。如要進一步瞭解存取權控管，請參閱 BigQuery 說明文件中的[這篇文章](#)。

查詢公開資料集前，請確認服務帳戶至少具備 **bigquery.user** 角色。在 Cloud Shell 中執行下列指令，將 bigquery.user 角色指派給服務帳戶：

```
gcloud projects add-iam-policy-binding ${GOOGLE_CLOUD_PROJECT} --member "serviceAccount:my-bigquery-sa@${GOOGLE_CLOUD_PROJECT}.iam.gserviceaccount.com" --role "roles/bigquery.user"
```

您可以執行下列指令，確認服務帳戶已獲派使用者角色：

```
gcloud projects get-iam-policy $GOOGLE_CLOUD_PROJECT
```

步驟 6 · 約 2 分鐘

6. 安裝 Node.js 專用的 BigQuery 用戶端程式庫

首先，建立 `BigQueryDemo` 資料夾並前往該資料夾：

```
mkdir BigQueryDemo
cd BigQueryDemo
```

接著，建立 Node.js 專案，用於執行 BigQuery 用戶端程式庫範例：

```
npm init -y
```

您應該會看到建立的 Node.js 專案：

```
{
  "name": "BigQueryDemo",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC"
}
```

安裝 BigQuery 用戶端程式庫：

```
npm install --save @google-cloud/bigquery
```

```
npm notice created a lockfile as package-lock.json. You should commit this file.
npm WARN BigQueryDemo@1.0.0 No description
npm WARN BigQueryDemo@1.0.0 No repository field.
+ @google-cloud/bigquery@2.1.0
added 69 packages from 105 contributors and audited 147 packages in 4.679s
found 0 vulnerabilities
```

現在，您已準備好使用 BigQuery Node.js 用戶端程式庫！

7. 查詢莎士比亞的作品

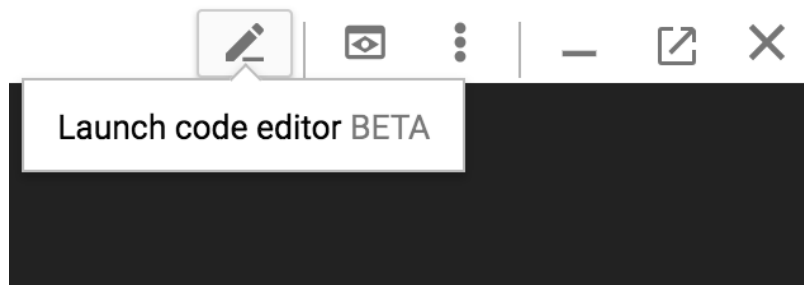
公開資料集是儲存在 BigQuery 中且可供一般大眾使用的任何資料集。有其他許多公開資料集可供您查詢，其中有些也是由 Google 託管，但有更多是由第三方託管，詳情請參閱「[公開資料集](#)」頁面。

除了公開資料集外，BigQuery 還提供了有限數量的[範例資料表](#)供您查詢。這些資料表位於 `bigquery-public-data:samples dataset` 中。其中一個資料表稱為 `shakespeare.`，內含莎士比亞作品的文字索引，可指出每個文字在各語料庫中出現的次數。

在這個步驟中，您將查詢 `shakespeare` 資料表。

注意：您可以在 BigQuery 主控台中[這裡](#)查看 `shakespeare` 資料表的詳細資料。

首先，從 Cloud Shell 右上方開啟程式碼編輯器：



在 `BigQueryDemo` 資料夾中建立 `queryShakespeare.js` 檔案：

```
touch queryShakespeare.js
```

前往 `queryShakespeare.js` 檔案，並插入下列程式碼：

```

'use strict';

function main() {

    // Import the Google Cloud client library
    const {BigQuery} = require('@google-cloud/bigquery');

    async function queryShakespeare() {
        // Queries a public Shakespeare dataset.

        // Create a client
        const bigqueryClient = new BigQuery();

        // The SQL query to run
        const sqlQuery = `SELECT word, word_count
            FROM `bigquery-public-data.samples.shakespeare`
            WHERE corpus = @corpus
            AND word_count >= @min_word_count
            ORDER BY word_count DESC`;

        const options = {
            query: sqlQuery,
            // Location must match that of the dataset(s) referenced in the query.
            location: 'US',
            params: {corpus: 'romeoandjuliet', min_word_count: 250},
        };

        // Run the query
        const [rows] = await bigqueryClient.query(options);

        console.log('Rows:');
        rows.forEach(row => console.log(row));
    }

    queryShakespeare();
}

main();

```

請花一到兩分鐘研究程式碼，瞭解如何查詢資料表。

回到 Cloud Shell，執行應用程式：

```
node queryShakespeare.js
```

畫面上應會顯示字詞清單和出現次數：

Rows:

```
{ word: 'the', word_count: 614 }
{ word: 'I', word_count: 577 }
{ word: 'and', word_count: 490 }
{ word: 'to', word_count: 486 }
{ word: 'a', word_count: 407 }
{ word: 'of', word_count: 367 }
{ word: 'my', word_count: 314 }
{ word: 'is', word_count: 307 }
...
```

注意：如果這個 Node.js 程式碼無法運作，請驗證您在「**驗證 API 要求**」步驟中執行的操作。

使用下列指令驗證 `GOOGLE_APPLICATION_CREDENTIALS` 環境變數的值：

```
echo $GOOGLE_APPLICATION_CREDENTIALS
```

應該會輸出值 `"~/key.json"`。

如果有的話，請使用下列指令，確認服務帳戶已建立並位於 `"~/key.json"`：

```
cat "~/key.json"
```

您應該會看到類似下列內容：

```
{
  "type": "service_account",
  "project_id": "PROJECT_ID",
  "private_key_id": "ff31939192529e07f42e4535fb20bb029def1276",
  "Private_key":...
```

如果沒有，請返回「**驗證 API 要求**」步驟。

步驟 8 · 約 5 分鐘

8. 查詢 GitHub 資料集

為進一步熟悉 BigQuery，您現在要對 [GitHub 公開資料集](#) 發出查詢。您可以在 GitHub 上找到最常見的提交訊息。您也會使用 BigQuery 的網頁版 UI 預覽及執行臨時查詢。

如要查看資料，請在 BigQuery 網頁版 UI 中開啟 GitHub 資料集：

https://console.cloud.google.com/bigquery?p=bigquery-public-data&d=github_repos&t=commits&page=table

如要快速預覽資料外觀，請按一下「預覽」分頁標籤：

The screenshot shows the BigQuery console interface. On the left, the 'Resources' panel lists various datasets, with 'commits' under 'github_repos' selected. The main area displays the 'commits' table in 'Preview' mode. A red box highlights the 'Preview' button, and a red arrow points to it with the text 'Use Preview instead of SELECT *'. Below the table, there are buttons for 'Run query', 'Save query', 'Save view', and 'Options'.

Row	commit	tree
1	65359026795557aa2d210f200e286e77f64ee274	5c99924b3cd55372a10d4fb7a76896332568c5c6
2	5276005e2270730e92dfd3afa1b140230715687	665f0f7190a25878277c999e7794ca3aa33e353f
3	6c9aca4718926b42e1c794cb64423a58a687b0f1	4b825dc642cb6eb9a060e54bf8d69288fbee4904
4	7aeaec60718b73dac8892a8016ca7fc746b062c1	da2e033840dad23bf2867a2987f42e88dcf71071
5	5ee2e5f814b0c4f136ef5a75d3a56fa814a0e099	faee088f844e55000c0e9fcf45eaf03bdb2cb18f

在 `BigQueryDemo` 資料夾中建立 `queryGitHub.js` 檔案：

```
touch queryGitHub.js
```

前往 `queryGitHub.js` 檔案，並插入下列程式碼：

```

'use strict';

function main() {

    // Import the Google Cloud client library
    const {BigQuery} = require('@google-cloud/bigquery');

    async function queryGitHub() {
        // Queries a public GitHub dataset.

        // Create a client
        const bigqueryClient = new BigQuery();

        // The SQL query to run
        const sqlQuery = `SELECT subject AS subject, COUNT(*) AS num_duplicates
FROM \`bigquery-public-data.github_repos.commits\`
GROUP BY subject
ORDER BY num_duplicates
DESC LIMIT 10`;

        const options = {
            query: sqlQuery,
            // Location must match that of the dataset(s) referenced in the query.
            location: 'US',
        };

        // Run the query
        const [rows] = await bigqueryClient.query(options);

        console.log('Rows:');
        rows.forEach(row => console.log(`${row.subject}: ${row.num_duplicates}`));
    }

    queryGitHub();
}

main();

```

請花一兩分鐘研究程式碼，瞭解如何查詢表格中最常見的提交訊息。

回到 Cloud Shell，執行應用程式：

```
node queryGitHub.js
```

畫面上應會顯示提交訊息清單和出現次數：

Rows:

Update README.md: 2572220

: 1985733

Initial commit: 1940228

Mirroring from Micro.blog.: 646772

update: 592520

Update data.json: 548730

Update data.js: 548354

...

步驟 9 · 約 3 分鐘

9. 快取和統計資料

執行查詢時，BigQuery 會快取結果。因此，後續的相同查詢所需時間會大幅減少。您可以使用查詢選項停用快取。BigQuery 也會追蹤查詢的某些統計資料，例如建立時間、結束時間和處理的位元組總數。

在這個步驟中，您將停用快取，並顯示一些查詢的統計資料。

前往 `BigQueryDemo` 資料夾中的 `queryShakespeare.js` 檔案，並將程式碼替換成以下程式碼：

```

'use strict';

function main() {

    // Import the Google Cloud client library
    const {BigQuery} = require('@google-cloud/bigquery');

    async function queryShakespeareDisableCache() {
        // Queries the Shakespeare dataset with the cache disabled.

        // Create a client
        const bigqueryClient = new BigQuery();

        const sqlQuery = `SELECT word, word_count
            FROM `bigquery-public-data.samples.shakespeare`
            WHERE corpus = @corpus
            AND word_count >= @min_word_count
            ORDER BY word_count DESC`;

        const options = {
            query: sqlQuery,
            // Location must match that of the dataset(s) referenced in the query.
            location: 'US',
            params: {corpus: 'romeoandjuliet', min_word_count: 250},
            useQueryCache: false,
        };

        // Run the query as a job
        const [job] = await bigqueryClient.createQueryJob(options);
        console.log(`Job ${job.id} started.`);

        // Wait for the query to finish
        const [rows] = await job.getQueryResults();

        // Print the results
        console.log('Rows:');
        rows.forEach(row => console.log(row));

        // Print job statistics
        console.log('JOB STATISTICS:')
        console.log(`Status: ${job.metadata.status.state}`);
        console.log(`Creation time: ${job.metadata.statistics.creationTime}`);
        console.log(`Start time: ${job.metadata.statistics.startTime}`);
        console.log(`Statement type: ${job.metadata.statistics.query.statementType}`);
    }
    queryShakespeareDisableCache();
}

main();

```

請注意以下幾點：首先，在 `options` 物件中將 `UseQueryCache` 設為 `false`，即可停用快取。其次，您從工作物件存取查詢的統計資料。

回到 Cloud Shell，執行應用程式：

```
node queryShakespeare.js
```

畫面上應會顯示提交訊息清單和出現次數。此外，您也應該會看到一些查詢統計資料：

```
Rows:
{ word: 'the', word_count: 614 }
{ word: 'I', word_count: 577 }
{ word: 'and', word_count: 490 }
{ word: 'to', word_count: 486 }
{ word: 'a', word_count: 407 }
{ word: 'of', word_count: 367 }
{ word: 'my', word_count: 314 }
{ word: 'is', word_count: 307 }
{ word: 'in', word_count: 291 }
{ word: 'you', word_count: 271 }
{ word: 'that', word_count: 270 }
{ word: 'me', word_count: 263 }
JOB STATISTICS:
Status: RUNNING
Creation time: 1554309220660
Start time: 1554309220793
Statement type: SELECT
```

10. 將資料載入 BigQuery

如要查詢自己的資料，請先將資料載入 BigQuery。BigQuery 支援從許多來源載入資料，例如 Google Cloud Storage、其他 Google 服務，或是可讀取的本機來源。你甚至可以串流播放資料。如要瞭解詳情，請參閱「[將資料載入 BigQuery](#)」頁面。

在這個步驟中，您會將儲存在 Google Cloud Storage 中的 JSON 檔案載入 BigQuery 資料表。JSON 檔案位於：`gs://cloud-samples-data/bigquery/us-states/us-states.json`

如要查看 JSON 檔案內容，可以使用 `gsutil` 指令列工具在 Cloud Shell 中下載：

```
gsutil cp gs://cloud-samples-data/bigquery/us-states/us-states.json .
```

```
Copying gs://cloud-samples-data/bigquery/us-states/us-states.json...
/ [1 files][ 2.0 KiB/ 2.0 KiB]
Operation completed over 1 objects/2.0 KiB.
```

您可以看到其中包含美國各州清單，每個州都是獨立成行的 JSON 物件：

```
less us-states.json
```

```
{"name": "Alabama", "post_abbr": "AL"}
{"name": "Alaska", "post_abbr": "AK"}
...
```

如要將這個 JSON 檔案載入 BigQuery，請在 `BigQueryDemo` 資料夾中建立 `createDataset.js` 檔案和 `loadBigQueryJSON.js` 檔案：

```
touch createDataset.js
touch loadBigQueryJSON.js
```

安裝 Google Cloud Storage Node.js 用戶端程式庫：

```
npm install --save @google-cloud/storage
```

前往 `createDataset.js` 檔案，並插入下列程式碼：

```
'use strict';

function main() {
  // Import the Google Cloud client libraries
  const {BigQuery} = require('@google-cloud/bigquery');

  async function createDataset() {
    const datasetId = "my_states_dataset3";

    const bigqueryClient = new BigQuery();

    // Specify the geographic location where the dataset should reside
    const options = {
      location: 'US',
    };

    // Create a new dataset
    const [dataset] = await bigqueryClient.createDataset(datasetId, options);
    console.log(`Dataset ${dataset.id} created.`);
  }

  createDataset();
}

main();
```

接著前往 `loadBigQueryJSON.js` 檔案，並插入下列程式碼：

```

'use strict';

function main() {
  // Import the Google Cloud client libraries
  const {BigQuery} = require('@google-cloud/bigquery');
  const {Storage} = require('@google-cloud/storage');

  const datasetId = "my_states_dataset3";
  const tableId = "my_states_table";

  async function createTable(datasetId, tableId) {
    // Creates a new table

    // Create a client
    const bigqueryClient = new BigQuery();

    const options = {
      location: 'US',
    };

    // Create a new table in the dataset
    const [table] = await bigqueryClient
      .dataset(datasetId)
      .createTable(tableId, options);

    console.log(`Table ${table.id} created.`);
  }

  async function loadJSONFromGCS(datasetId, tableId) {
    // Import a GCS file into a table with manually defined schema.

    // Instantiate clients
    const bigqueryClient = new BigQuery();
    const storageClient = new Storage();

    const bucketName = 'cloud-samples-data';
    const filename = 'bigquery/us-states/us-states.json';

    // Configure the load job.
    const metadata = {
      sourceFormat: 'NEWLINE_DELIMITED_JSON',
      schema: {
        fields: [
          {name: 'name', type: 'STRING'},
          {name: 'post_abbr', type: 'STRING'},
        ],
      },
      location: 'US',
    };

    // Load data from a Google Cloud Storage file into the table
  }
}

```

```

const [job] = await bigqueryClient
  .dataset(datasetId)
  .table(tableId)
  .load(storageClient.bucket(bucketName).file(filename), metadata);

// load() waits for the job to finish
console.log(`Job ${job.id} completed.`);

// Check the job's status for errors
const errors = job.status.errors;
if (errors && errors.length > 0) {
  throw errors;
}
}

// createDataset(datasetId);
createTable(datasetId, tableId);
loadJSONFromGCS(datasetId, tableId);
}

main();

```

請花一兩分鐘研究程式碼如何載入 JSON 檔案，並在資料集中建立資料表 (含結構定義)。

回到 Cloud Shell，執行應用程式：

```

node createDataset.js
node loadBigQueryJSON.js

```

系統會在 BigQuery 中建立資料集和資料表：

```

Table my_states_table created.
Job [JOB ID] completed.

```

如要確認資料集是否已建立，請前往 BigQuery 網頁版 UI。您應該會看到新的資料集和資料表。切換至資料表的「預覽」分頁，即可查看實際資料：

us_states_dataset

us_states_table

bigquery-samples

Run query

Save query

us_states_table

SchemaDetailsPreview

Row	name	post_abbr
1	Alabama	AL
2	Alaska	AK
3	Arizona	AZ
4	Arkansas	AR
5	California	CA
6	Colorado	CO
7	Connecticut	CT
8	Delaware	DE
9	Florida	FL

11. 恭喜！

您已瞭解如何使用 Node.js 存取 BigQuery！

清除所用資源

如何避免系統向您的 Google Cloud Platform 帳戶收取您在本快速入門導覽課程中所用資源的相關費用：

- 前往 [Cloud Platform Console](#)。
- 選取要關閉的專案，然後按一下頂端的「刪除」，系統就會排定刪除專案的時間。

瞭解詳情

- Google BigQuery：<https://cloud.google.com/bigquery/docs/>
- Google Cloud Platform 上的 Node.js：<https://cloud.google.com/nodejs/>
- Google BigQuery Node.js 用戶端程式庫：<https://github.com/googleapis/nodejs-bigquery>

授權

這項內容採用的授權為 Creative Commons 姓名標示 2.0 通用授權。
